Erpta — ERP com banking e crédito inteligente

O ERP que lê a operação da empresa e encontra o crédito certo.
Entrega final: Etapa 4 (produção) e Fase 2 (portal do cliente).

Murilo · Lucas · Gabriel · João Vivacqua · Vinicius · João Baungartner

Quem fala o quê

Murilo
Problema, ErpIa
slides 3–4

Lucas
Arquitetura e Etapa 4
slides 5–7

Gabriel
Jornada do cliente, demo
slides 8–9, 19

João Vivacqua
Portal e crédito integrado
slides 10–11

Vinicius
Assistente, equipe
slides 12–14

João Baungartner
Qualidade, segurança, isolamento, padrões
slides 15–18

Próximos passos e perguntas: todos.

O problema

A pequena empresa opera em **planilhas e três sistemas**: o ERP registra vendas e contas, o banco guarda o dinheiro e o crédito chega tarde, caro e sem contexto.

- O dono só descobre o **vale de caixa** quando o boleto vence.
- O banco não enxerga recebíveis, estoque nem sazonalidade, então o crédito vira **cheque especial**.
- Do lado do banco: cadastro, score, decisão de crédito e permissões precisam ser **auditáveis** e escalar.

O que nos propusemos

Um único sistema que **registra a operação**, **movimenta o dinheiro** e **oferece o crédito certo no momento certo**, com regras explicáveis e IA só onde ela ajuda.

A ErpIa

ERP + conta PJ + crédito num portal só, com backoffice para o banco.

- Vendas, recebíveis, estoque e contas a pagar.
- PIX, boletos e extrato da conta PJ.
- Em cada tela, o **crédito que resolve** aquele problema.
- Assistente que responde com os números da empresa.

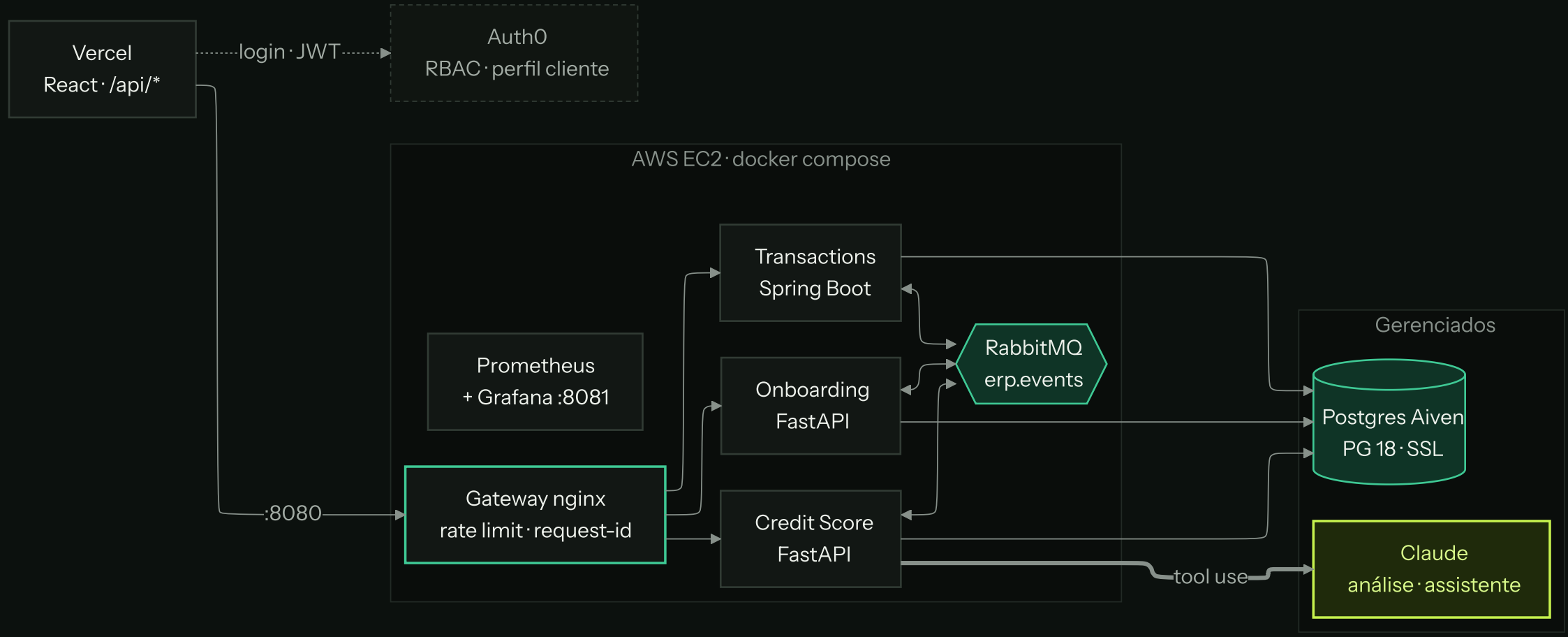
✦ Faísca

A estrela lima marca só o que a IA infere: insights, oportunidades e valores pré-aprovados.



Landing em produção: erp-banking-front.vercel.app

Arquitetura final



Etapa 4 · gateway e eventos

API Gateway nginx

- Porta pública única (**8080**): `/onboarding`, `/score`, `/transactions`.
- Rate limit **50 req/s** por cliente + teto por IP → **429** com `Retry-After`.
- `X-Request-ID` em todos os logs; serviço fora → **503** JSON.
- `/metrics` e `/actuator` fora da internet.

Assíncrono · RabbitMQ

- Exchange `erp.events` (topic, durável), envelope com `event_id`.
- `customer.created` abre a conta; `credit.decided` chega ao Score.
- Fase 2: `onboarding_submitted`, `analysis_completed`, `onboarding_reviewed`.
- Consumidores **idempotentes**; publicar nunca derruba o HTTP.

Etapa 4 · em produção

9/9	7/7	6	14
containers saudáveis na EC2 3.235.6.136	alvos do Prometheus up	dashboards Grafana provisionados + Portal	alertas (10 da Etapa 4 + 4 do portal)

Deploy
CI publica imagens no GHCR (`:latest` e `:sha`); EC2 com `docker compose` ; front na **Vercel** com rewrites `/api/*` .

Banco gerenciado
Aiven, Postgres 18 com SSL. Pools de 2+1 conexões por serviço Python e 3 no Java, dentro do limite de 20.

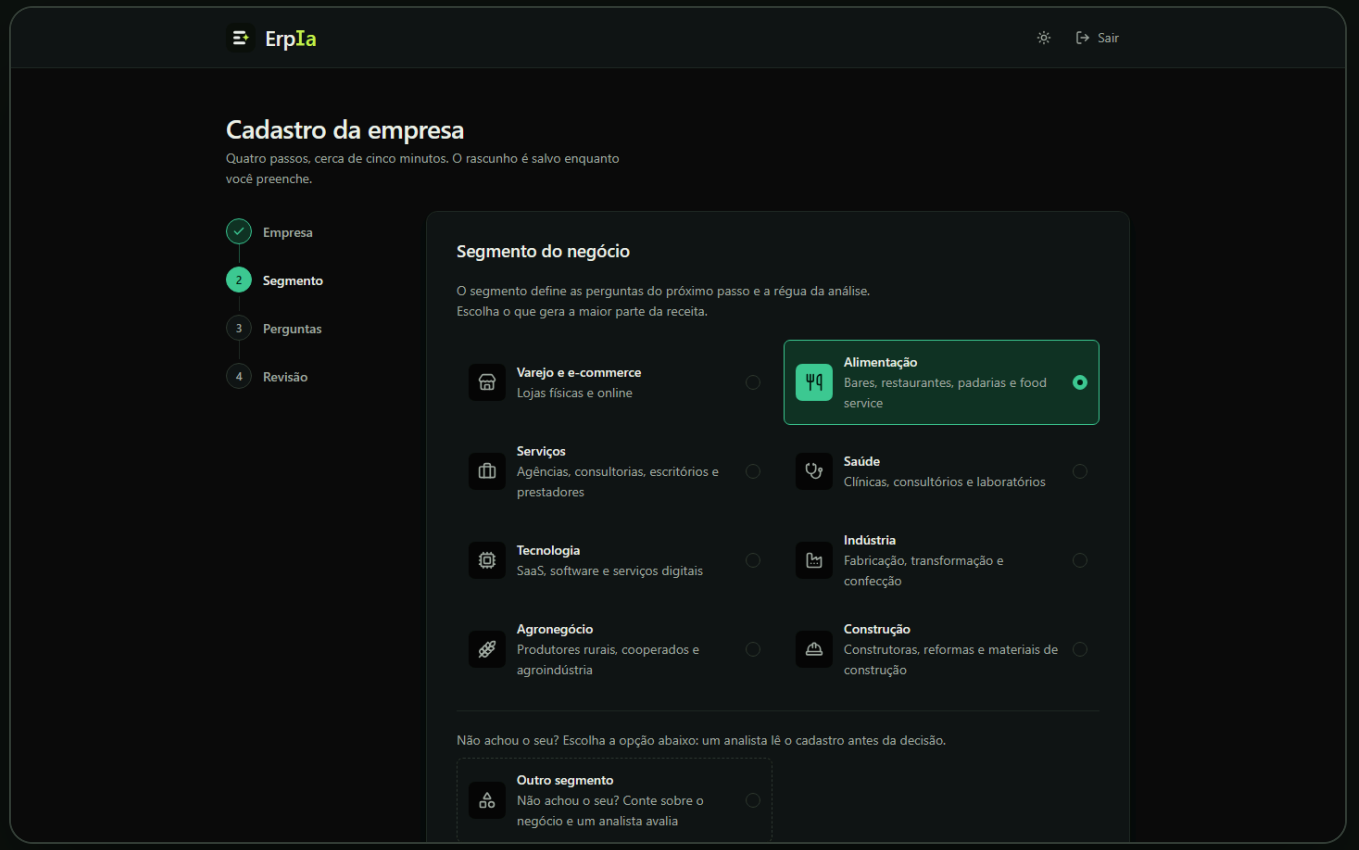
Observabilidade
Métricas de negócio (decisões de crédito, análises, mensagens do assistente, tokens) e saúde em `/monitoramento` .

Jornada do cliente

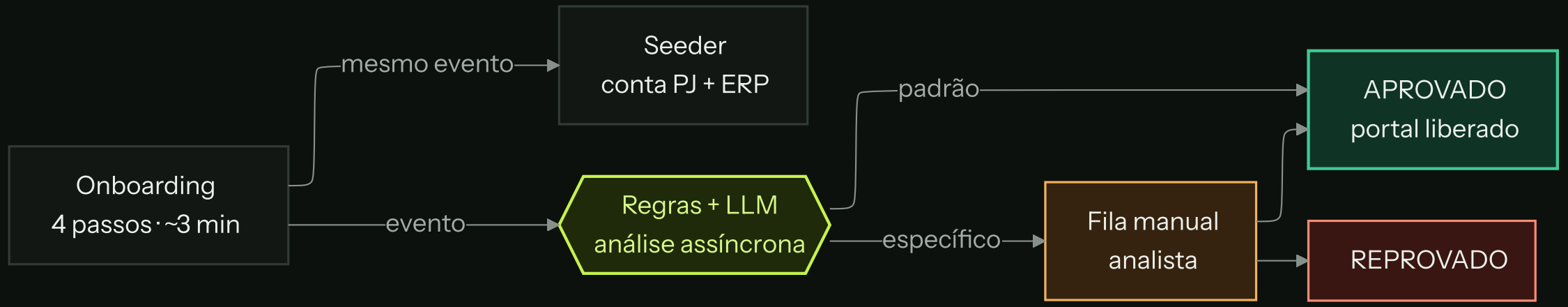
~3 min
do cadastro ao envio

9
segmentos com perguntas próprias

- Auth0 signup; a Post-Login Action dá o perfil **cliente**.
- 4 passos: empresa, **segmento**, perguntas, revisão.
- 8 a 14 campos por segmento; **rascunho salvo** a cada passo.
- Observações livres sobre o negócio vão para a análise.



Análise: regras + LLM



Regras, sempre

KPIs, score 0–1000, faixa de risco, insights e oportunidades com valor. Explicável e determinístico.

✦ LLM, quando ajuda

Claude lê dados e observações e devolve JSON: enquadramento, confiança, riscos. Timeout de 20 s; se falhar, ficam as regras.

Fila manual

Segmento "outro", perfil específico, confiança < 0,6 ou regra crítica → analista decide em `/analises`.

Portal · visão geral

Aurora Pães, em produção

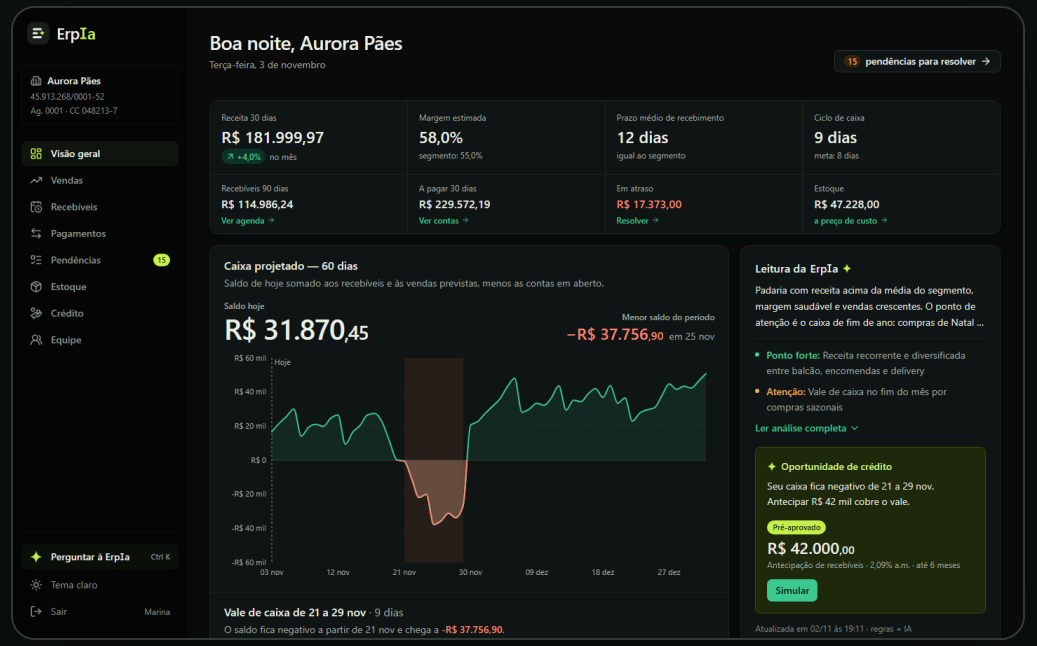
R$ 31.870,45
saldo da conta PJ

R$ 181.999,97
receita 30 dias

742
score · faixa MÉDIO

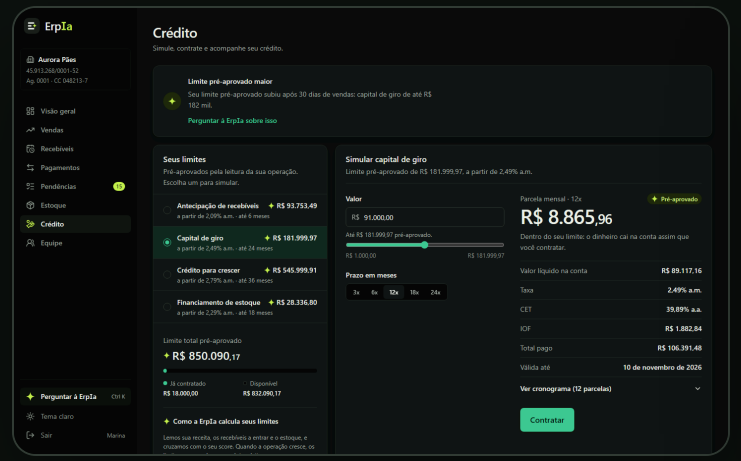
−R$ 38.324,50
vale de caixa, 20 a 29/11

✦ Antecipação pré-aprovada cobre o vale · 15 pendências abertas

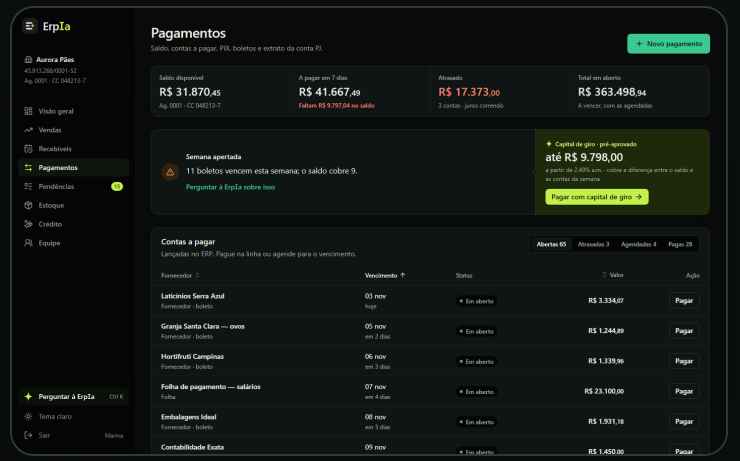


Captura com dados de demonstração (mocks); os números ao lado são os da produção.

Crédito em cada tela







Crédito · simulador com parcelas Price, IOF e CET; limites por produto.

Recebíveis · agenda por adquirente; selecionar e antecipar.

Pagamentos · PIX, boleto e agendamento; pagar com capital de giro quando o saldo não cobre.

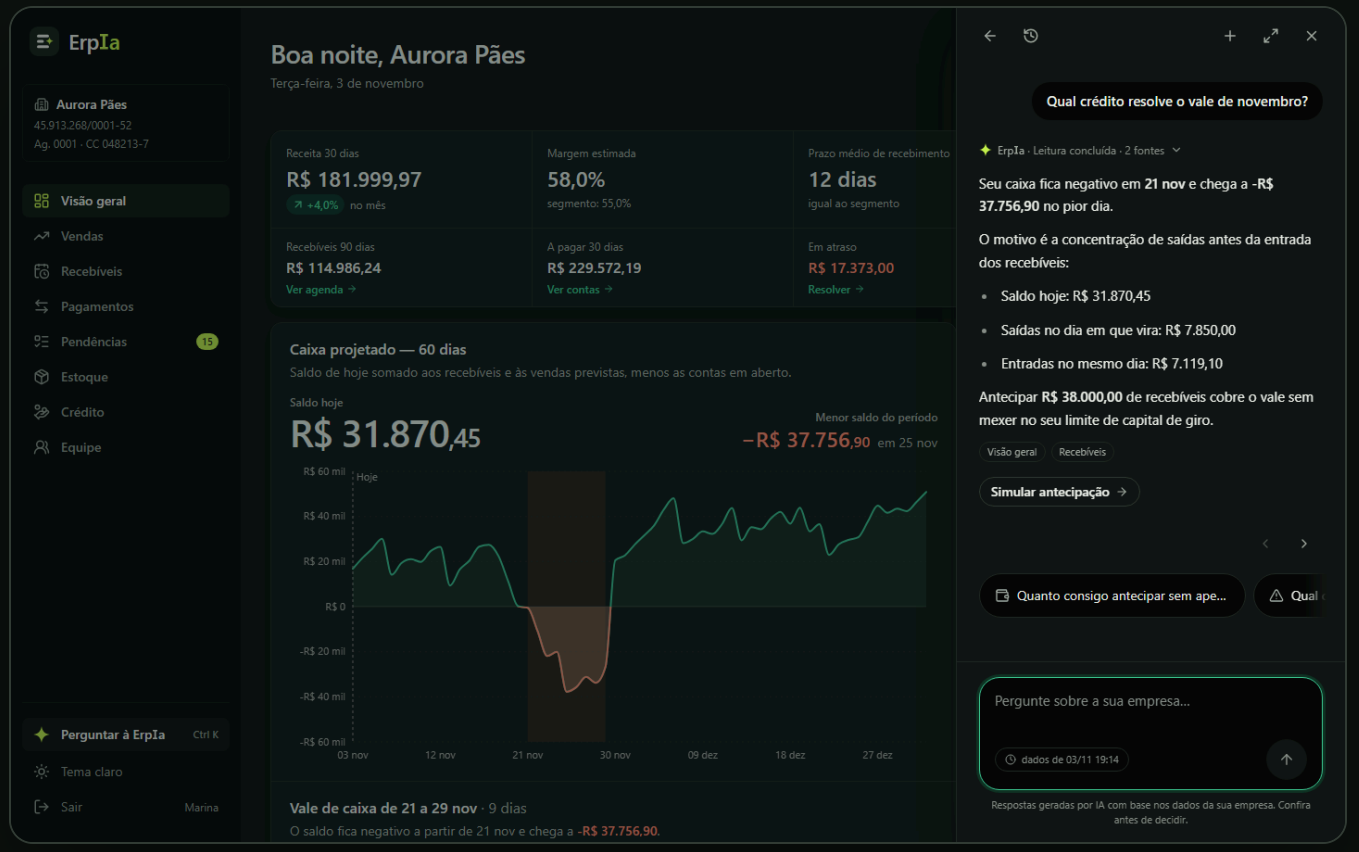
✦ Precificação explicável — taxa base pela faixa (BAIXO 1,69% · MÉDIO 2,49% · ALTO 3,49% a.m.), ajuste por produto e limite pelo lastro: 80% dos recebíveis, 60% do estoque, 3× a receita.

Assistente ErpIa

✦ Tool use seguro

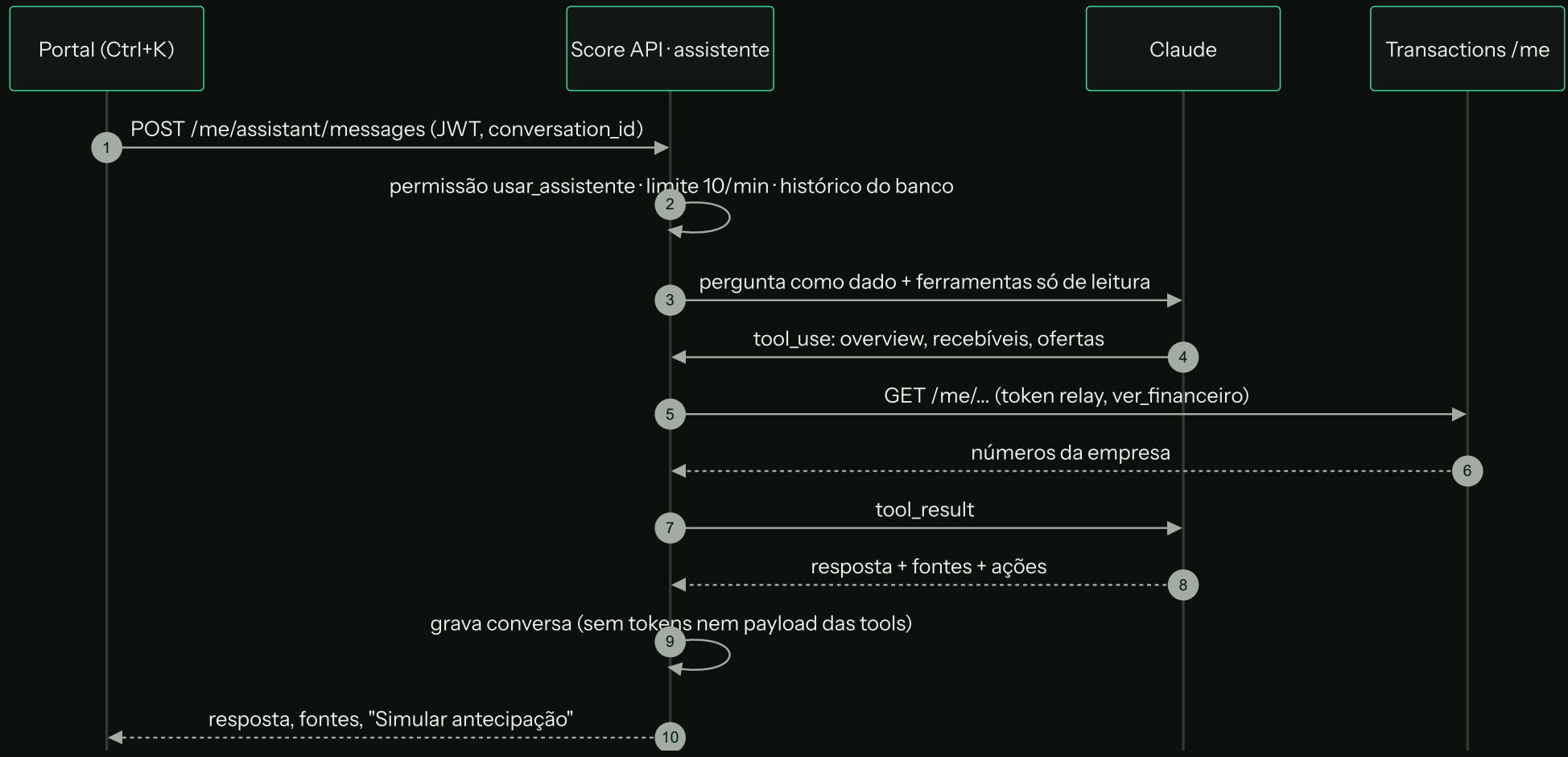
Claude chama só ferramentas de **leitura e simulação** do Transactions, com o token do próprio usuário. Nunca paga nem contrata.

- Até 6 chamadas de ferramenta por resposta; 10 mensagens/min.
- Responde com **fontes** e **ações** ("Simular antecipação").
- **Histórico no banco**: retomar, apagar uma ou todas (LGPD).
- **E2E aprovado** em produção com Claude real usando tools.



Ctrl+K abre o painel em qualquer tela do portal.

Assistente · uma pergunta por dentro



Equipe e permissões

O dono convida funcionários e decide **o que cada um pode fazer**. No Auth0 todos são `cliente`; as permissões finas ficam no nosso banco e o **back aplica**.

ver_financeiro · pagar · aprovar_pagamentos · pedir_credito · contratar_credito · gerenciar_estoque · gerenciar_equipe · usar_assistente

Papel	Preset
DONO/ADMIN	todas
FINANCEIRO	pagar, aprovar, pedir crédito, assistente
OPERADOR	pagar até um limite, estoque
LEITURA	só ver o financeiro

Aprovação de pagamentos: acima do limite o pagamento fica pendente, sem debitar, até outra pessoa aprovar. Convite por link de uso único (hash, 7 dias).

Qualidade e carga

97–99%	426 ms	0	~280 req/s
cobertura nos 4 repos (CI falha abaixo de 80%)	p95 do fluxo de negócio autenticado, 30 VUs	respostas 5xx em todos os cenários	sustentados pelo gateway no stress

CI em todo PR: lint, format, typecheck, testes com cobertura, build Docker e publish no GHCR. Merge só com CI verde; Conventional Commits.

k6 na EC2: smoke, load, rate-limit, stress e spike (p95 < 12 ms) + fluxo de negócio com token real do Auth0 (cliente → score → conta → crédito). Os 429 são o rate limit funcionando: todo o tráfego sai de um IP.

Segurança

Identidade

Auth0 com RBAC: perfis de staff e **cliente**, Post-Login Action para signup self-service. Permissões no JWT, checadas em cada serviço.

Entre serviços

Token relay: o JWT do usuário segue nas chamadas síncronas. Gateway como única porta, rate limit e métricas internas.

Dados e eventos

Outbox transacional no Onboarding: evento gravado na mesma transação e republicado se o broker cair. Consumidores idempotentes.

✦ IA

Texto do usuário tratado como dado (proteção contra prompt injection), ferramentas só de leitura, sem tokens no histórico.

Segredos

Fora do git: `.env` da EC2, variáveis da Vercel e do Auth0. A chave da Anthropic nunca vai para log nem resposta.

Conexões

Postgres com SSL na Aiven; HTTPS na Vercel; headers de segurança; convites guardados só como hash.

Isolamento entre empresas

28	11	0
rotas e fluxos revisados nos 3 serviços + gateway	correções mergeadas	vazamentos entre empresas

Serviço	Corrigido
Transactions	recebíveis de outra empresa na simulação → sempre 404
Transactions	`receivable_ids` enorme dava 500 → máx. 200, sem duplicata
Transactions	agendamento sem limite de data → até hoje + 365 dias
Transactions	CPF/e-mail em log de exceção → só `event_id` (LogSafe)
Onboarding	token do convite no access log → mascarado
Onboarding	limite NaN/Infinity dava 500 → 422

Serviço	Corrigido
Onboarding	documento duplicado no 409 → só os 4 últimos dígitos
Onboarding	consumers logavam os valores recebidos → removido
Score	ex-membro lia conversas da empresa antiga → filtro pela atual
Score	CNPJ e respostas em log dos consumers → removido
Gateway	token do convite no log do nginx → `/invites/***/accept`

Teste de 404 entre empresas: corpo idêntico ao de id inexistente, nenhum dado alheio, nada muda no banco.

Padrões de projeto

Padrão	Onde
Strategy + Factory	Score: estratégias de score; `BusinessAnalyzer` com `RuleBasedAnalyzer` e `LlmAnalyzer`
API Gateway	nginx: ponto único de entrada, rate limit, request-id
Publish/Subscribe	RabbitMQ `erp.events`, vários consumidores por evento
Transactional Outbox	Onboarding: `outbox_events` + relay com `FOR UPDATE SKIP LOCKED`
Consumidor idempotente	`event_id` único; seeder e análise não refazem trabalho
Circuit Breaker + Retry	Transactions → Score (timeout, retry, 503 claro)
Token relay	serviços e ferramentas do assistente repassam o JWT do usuário
Repository · Database per service	SQLAlchemy e Spring Data JPA; um schema por serviço

Demo · roteiro

1 · Landing e cadastro

Criar conta

Signup no Auth0, perfil cliente pela Action, onboarding de alimentação em ~3 min.

2 · Análise

Regras + IA

Status com timeline; um caso "outro" cai na fila e o analista aprova em /analises.

3 · Portal

Aurora Pães

Visão geral, vale de caixa, antecipação pré-aprovada, simulador e um PIX.

4 · Assistente

Ctrl+K

"Qual crédito resolve o vale de novembro?" com ferramentas e fontes.

5 · Equipe

Convite

Operador com limite: o pagamento acima dele fica pendente de aprovação.

6 · Operação

Grafana

Dashboards Portal e k6, métricas de negócio e do assistente.

Próximos passos

Plataforma

- **TLS no gateway** (domínio + Let's Encrypt) e fechar a porta HTTP.
- **DLQ** e retentativas com backoff nos consumidores.
- **M2M** (client credentials) entre serviços no lugar do token relay.
- Réplicas atrás do gateway; RabbitMQ gerenciado; tracing com OpenTelemetry.

Produto

- Open Finance para importar extratos e recebíveis reais.
- Envio de convites e alertas por e-mail.
- Assistente com streaming e ações confirmadas pelo usuário.
- Contratação de crédito com assinatura e formalização.

Perguntas

Front **erp-banking-front.vercel.app** · Gateway **3.235.6.136:8080** · Grafana **3.235.6.136:8081**

Código em github.com/murilozgodoy/erp-banking-* · documentação em erp-banking-front/docs/